

Python Practice Handout

Parameters and return values

1. What is a parameter

A parameter is a way to pass information into a function. The hello function from before always printed the same name. With a parameter you can hand it a different name each time.

Parameters go inside the brackets after the function name. Here the parameter is called name.

```
def hello(name):  
    print(f"Hello {name}")
```

Note the f before the quote. The parameter is used inside the string just like any other variable.

2. Passing a value in

When you call the function, put the value inside the brackets. The same function now gives a different result each time.

```
def hello(name):  
    print(f"Hello {name}")  
  
hello("Nick")  
hello("Sara")  
hello("John")
```

Output:

```
Hello Nick  
Hello Sara  
Hello John
```

3. You must pass something in

If the function expects a parameter, calling it with empty brackets is an error. Python tells you exactly which one is missing.

This does not work:

```
hello()
```

Error:

```
TypeError: hello() missing 1 required  
positional argument: 'name'
```

4. More than one parameter

Separate parameters with a comma. This function takes two numbers and prints their total. Call it with two values, also separated by a comma.

```
def add_numbers(num1, num2):  
    print(num1 + num2)  
  
add_numbers(4, 8)  
add_numbers(3, 7)
```

Output:

```
12  
10
```

The order matters. The first value goes into num1 and the second into num2.

5. Try it yourself

Make a function called dog_info that takes a dog's age and name and prints a sentence about the dog. Try this on your own before looking below.

```
def dog_info(age, name):  
    print(f"Hi, my name is {name} and I am {age} years old")  
  
dog_info(4, "Sara")
```

Output:

```
Hi, my name is Sara and I am 4 years old
```

6. Returning information

Parameters pass information in. Returning passes information back out. Write return, then whatever you want to send back.

This function doubles a number. On its own it still prints nothing, because returning is not the same as printing.

```
def double(number):  
    return number * 2  
  
double(5)
```

Output:

(nothing)

7. Using what comes back

The call itself now stands for the value that came back. Print the call and you see the answer.

```
def double(number):  
    return number * 2  
  
print(double(5))
```

Output:

```
10
```

You can also store the returned value in a variable and use it later.

```
new_number = double(5)

print(new_number)
```

Output:

```
10
```

You can return any type: ints, floats, strings or Booleans.

8. Try it yourself

Make a function that returns a string in capital letters. This one needs looking up, which is a normal part of programming. Search the web for python uppercase string.

```
def uppercase(text):
    return text.upper()

print(uppercase("Nick"))
```

Output:

```
NICK
```

9. Putting it together

Functions and for loops work well together. Make a list of names, loop through it, and pass each name into the function.

```
def uppercase(text):
    return text.upper()

names = ["Nick", "Jane", "Sara"]

for name in names:
    print(uppercase(name))
```

Output:

```
NICK
JANE
SARA
```